

Homework Assignment 8

Decision Tree, Random Forest & XGBoost

Submission

Submit your ipynb file with a pdf of your assignment solution through the assignment submission portal on Canvas. No need to zip the files.

Datasets

Position_Salaries.csv: Used for Questions 1–4

(https://raw.githubusercontent.com/mh2t/CS6140/refs/heads/main/data/Position_Salaries.csv)

Data.csv: Used for Question 5

(<https://raw.githubusercontent.com/mh2t/CS6140/refs/heads/main/data/Data.csv>)

Reference Code

The key model-fitting lines you need to call are shown below.

Decision Tree Regression (Position_Salaries.csv):

```
regressor = DecisionTreeRegressor(random_state=0)
regressor.fit(X, y)
```

Random Forest Regression (Position_Salaries.csv):

```
regressor = RandomForestRegressor(n_estimators=10, random_state=0)
regressor.fit(X, y)
```

XGBoost Classification (Data.csv):

```
classifier = XGBClassifier()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

Notes: The regression models assume X is the Level column and y is Salary, trained on the full dataset. The XGBoost classifier uses an 80/20 train/test split with random_state=0, default hyperparameters and 10-fold cross-validation on the training set.

Question 1 — Decision Tree Regression Pipeline

Build a complete Decision Tree regression pipeline from scratch using Position_Salaries.csv.

- Write a function **load_position_data(filepath)** that reads the CSV, extracts X (Level) and y (Salary) as NumPy arrays, and prints a diagnostic summary: shape, data types, min/max/mean of each column. Do not hard-code column names or indices. Return X, y, and the full DataFrame.

- b) Write a function **train_decision_tree(X, y, random_state=0, **kwargs)** that accepts any DecisionTreeRegressor hyperparameters via ****kwargs** (e.g., `max_depth`, `min_samples_split`, `min_samples_leaf`). Fit the model on the full dataset, print the tree depth, number of leaves, and R^2 score, and return the fitted model.
- c) Write a function **plot_tree_regression(X, y, model, title, step=0.01)** that creates a high-resolution step plot by generating predictions on a fine `X_grid` (with the given step size). Plot the data points as a scatter and the model predictions as a line. Annotate the plot with the R^2 score and the tree depth. Also use `sklearn.tree.plot_tree` to produce a second figure showing the full decision tree structure. Include titles and axis labels on both figures.

Question 2 — Decision Tree Hyperparameter Tuning

Using your pipeline from Question 1, investigate how hyperparameters affect the Decision Tree's behavior on `Position_Salaries.csv`.

- a) Write a function **depth_experiment(X, y, max_depths)** that trains a DecisionTreeRegressor for each `max_depth` in the provided list (e.g., `[1, 2, 3, 4, 5, None]` where `None` means unlimited). For each, record the R^2 , the number of leaves, and the prediction for level 6.5. Plot all models side-by-side in a 2x3 subplot grid using your `plot_tree_regression` function from Q1 (without the tree structure figure). Print a summary table of `max_depth` vs. R^2 vs. number of leaves vs. prediction at 6.5. In a code comment, discuss which `max_depth` provides the best balance between fitting the data and avoiding overfitting.
- b) Write a function **leaf_size_experiment(X, y, min_samples_values)** that trains a DecisionTreeRegressor for each `min_samples_leaf` value in the list (e.g., `[1, 2, 3, 4, 5]`). For each, record the R^2 , tree depth, and prediction at level 6.5. Plot the R^2 and tree depth vs. `min_samples_leaf` on the same chart (use twin y-axes). Return the results as a DataFrame.

Question 3 — Random Forest Regression Pipeline

Build a Random Forest regression pipeline and investigate the effect of `n_estimators` using `Position_Salaries.csv`.

- a) Write a function **train_random_forest(X, y, n_estimators=10, random_state=0, **kwargs)** that fits a RandomForestRegressor with any additional hyperparameters passed via ****kwargs**. Print the number of estimators, the average tree depth across all estimators (access via `model.estimators_`), and the R^2 score. Return the fitted model.
- b) Write a function **n_estimators_experiment(X, y, estimator_values)** that trains a Random Forest for each `n_estimators` value in the list (e.g., `[1, 5, 10, 50, 100, 500]`). For each, record the R^2 , the prediction at level 6.5, and the standard deviation of predictions across individual trees (use `model.estimators_` to get each tree's prediction at 6.5, then compute std). Plot two charts: (i) R^2 vs. `n_estimators`, and (ii) prediction at 6.5 with error bars showing the tree-level std vs. `n_estimators`. Return the results as a DataFrame.
- c) Write a function **plot_forest_vs_trees(X, y, model, n_trees_to_show=3)** that produces a subplot grid with `n_trees_to_show+1` panels. The first panel shows the full Random Forest prediction (using the ensemble's `predict` method). The remaining panels each show a single tree's prediction (access individual trees via `model.estimators_[i]`). Use a fine `X_grid` (step 0.01) for all panels. Title each panel accordingly ("Random Forest (n trees)" or "Tree i"). Include axis labels.

Question 4 — Model Comparison & Prediction

Compare Decision Tree and Random Forest regression head-to-head and build a prediction tool using `Position_Salaries.csv`.

- a) Write a function **`compare_models(X, y, dt_params, rf_params)`** that takes dictionaries of hyperparameters for each model (e.g., `dt_params={'max_depth': 4}` and `rf_params={'n_estimators': 100}`). Train both models, then: (i) plot both predictions side-by-side on a 1×2 subplot grid with the same y-axis scale, (ii) print a comparison table with columns: Model, R^2 , MAE, Max Error, Prediction at 6.5, and (iii) create an overlay plot showing both models' predictions on a single chart with different colors and a legend. Return both fitted models.
- b) Write a function **`predict_salaries(dt_model, rf_model, level_values, df)`** that takes both fitted models and a list of position levels (e.g., `[1.5, 3.5, 6.5, 8.5]`). For each level, predict the salary using both models, compute the absolute difference between the two predictions, and look up the nearest actual position and salary from the DataFrame. Print a table with columns: Level, Nearest Position, Actual Salary, DT Prediction, RF Prediction, Difference. In a code comment, discuss at which levels the two models disagree most and why.

Question 5 — XGBoost Classification Pipeline

Build a complete XGBoost classification pipeline using `Data.csv`, which contains cell nuclei measurements for breast cancer detection.

- a) Write a function **`load_and_split_classification(filepath, test_size=0.2, random_state=0)`** that reads `Data.csv`, separates features (all columns except the first and last) and target (last column), splits into training and test sets, and returns `X_train`, `X_test`, `y_train`, `y_test`. Print the shape of each set, the class distribution (count and percentage of each class) in both sets, and verify there are no missing values. Do not hard-code column names.
- b) Write a function **`train_and_evaluate_xgb(X_train, X_test, y_train, y_test, **kwargs)`** that trains an `XGBClassifier` with any hyperparameters passed via `**kwargs`. Evaluate it by printing: the confusion matrix, accuracy, precision, recall, and F1-score (from `sklearn.metrics`). Then run 10-fold cross-validation on the training set using `cross_val_score` with `scoring='accuracy'`, print the mean and standard deviation of the fold accuracies, and plot the individual fold scores as a bar chart. Return the fitted model.
- c) Write a function **`xgb_tuning_experiment(X_train, X_test, y_train, y_test, param_name, param_values)`** that trains an `XGBClassifier` for each value of a single hyperparameter (e.g., `param_name='n_estimators'`, `param_values=[10, 50, 100, 200, 500]` or `param_name='max_depth'`, `param_values=[2, 3, 4, 5, 6, 8]`). For each, record the test accuracy and 10-fold CV mean accuracy. Plot both metrics vs. the parameter value on the same chart with a legend. Return the results as a DataFrame. Demonstrate the function for both `n_estimators` and `max_depth`, and in a code comment recommend the best value for each.